

MARTOLA

*Modelo Avanzado de Rexistro e Tratamento de Organización de Leiras e
Agricultura*

Memoria do Proxecto Final de Ciclo

Ciclo Superior de Desenvolvemento de Aplicacións Multiplataforma (DAM)

Centro: CIFP A Carballeira Marcos Valcárcel

Autor: Iván Testa Vázquez

Curso académico: 2025/2026

Resumo

MARTOLA (Modelo Avanzado de Rexistro e Tratamento de Organización de Leiras e Agricultura) é unha aplicación multiplataforma desenvolvida con Flutter para facilitar a xestión de hortas e xardíns. A aplicación permite organizar hortas, plantas e especies, rexistrar a evolución dos cultivos, consultar información meteorolóxica asociada á localización, representar visualmente a disposición das plantas e xestionar tarefas básicas.

O proxecto segue un enfoque Local First, empregando SQLite como mecanismo principal de persistencia e servizos externos unicamente cando a funcionalidade require información remota, como a xeocodificación e a meteoroloxía. Desde o punto de vista arquitectónico aplícase un MVVM simplificado combinado co Repository Pattern e Provider para a xestión de estado, procurando unha separación clara entre interface, lóxica de presentación e acceso aos datos.

O desenvolvemento realizouse de forma incremental, partindo dun MVP e incorporando progresivamente persistencia, consumo de APIs, deseño responsive, migracións de base de datos e módulos adicionais. O resultado é un prototipo funcional e extensible que integra diferentes coñecementos propios do ciclo de Desenvolvemento de Aplicacións Multiplataforma e deixa preparadas futuras ampliacións como autenticación, sincronización cloud ou histórico meteorolóxico.

Palabras clave: Flutter, Dart, SQLite, Provider, MVVM, Repository Pattern, OpenWeather, aplicación multiplataforma, horticultura.

Índice

1. Introducción e contextualización	5
2. Estudo de mercado e alternativas existentes	6
3. Obxectivos e alcance	8
4. Análise de requisitos.....	10
5. Planificación e metodoloxía	13
6. Tecnoloxías e ferramentas	15
7. Arquitectura da aplicación	16
8. Deseño e persistencia dos datos	18
9. Deseño da interface e navegación.....	22
10. Implementación das funcionalidades.....	29
11. Probas e validación.....	30
12. Orzamento e viabilidade.....	33
13. Aspectos innovadores e valor achegado	34
14. Conclusións.....	35
15. Liñas futuras	37
16. Bibliografía e recursos.....	38
17. Anexos	39

1. Introducción e contextualización

O presente proxecto ten como finalidade o desenvolvemento de MARTOLA (Modelo Avanzado de Rexistro e Tratamento de Organización de Leiras e Agricultura), unha aplicación multiplataforma orientada á xestión de hortas e xardíns. O proxecto foi realizado como Traballo de Fin de Ciclo do Ciclo Superior de Desenvolvemento de Aplicacións Multiplataforma (DAM).

A idea de MARTOLA xorde da posibilidade de reunir nunha única aplicación diferentes ferramentas útiles para a organización e o seguimento dunha horta ou xardín. A xestión deste tipo de espazos pode implicar información diversa: as diferentes zonas ou hortas dispoñibles, as plantas cultivadas en cada unha delas, as datas de plantación, a evolución dos cultivos, as condicións meteorolóxicas, a distribución física das plantas ou as tarefas pendentes.

En lugar de tratar estes datos de maneira independente, MARTOLA propón centralizalos nunha aplicación cunha interface común e unha estrutura preparada para incorporar progresivamente novas funcionalidades.

O proxecto foi concibido desde o inicio cun enfoque multiplataforma, utilizando Flutter e Dart como tecnoloxías principais. A elección de Flutter permite compartir unha mesma base de código entre diferentes plataformas e adaptar a interface ás dimensións do dispositivo. A versión desenvolvida está orientada principalmente a Android e sistemas de escritorio, especialmente Windows e Linux.

Outro dos principios establecidos foi adoptar inicialmente un funcionamento local. Os datos propios da aplicación almacénanse mediante unha base de datos SQLite, polo que as funcionalidades principais de xestión non dependen dun servidor remoto. Esta decisión reduce a complexidade inicial do sistema e permite concentrar o desenvolvemento na funcionalidade e na arquitectura da aplicación, mantendo aberta a posibilidade de incorporar sincronización mediante servizos cloud nunha futura ampliación.

A aplicación complementa esta persistencia local mediante servizos externos cando a funcionalidade o require. En concreto, MARTOLA integra OpenWeather para consultar información meteorolóxica asociada ás hortas. Para isto almacénanse as coordenadas xeográficas de cada horta e utilízanse servizos de xeocodificación e consulta meteorolóxica. Os datos meteorolóxicos obtidos actualmente non se almacenan no histórico da base de datos, senón que son consultados cando resultan necesarios. Esta separación entre datos persistentes propios e información externa forma parte do alcance definido para a versión actual.

Durante o desenvolvemento incorporáronse diferentes módulos funcionais. Entre eles atópanse a xestión de hortas, a xestión das plantas asociadas a cada horta, un catálogo local de especies, o rexistro da evolución das plantas, a consulta meteorolóxica, un deseñador visual para representar a disposición das plantas dentro dunha horta e un sistema básico de xestión de tarefas.

Desde o punto de vista técnico, o proxecto tamén pretende aplicar conceptos propios do desenvolvemento de software máis alá da simple implementación das pantallas. MARTOLA utiliza unha arquitectura MVVM simplificada combinada co Repository Pattern, xestión de estado mediante Provider, separación entre acceso a datos e interface de usuario, repositorios abstractos e implementacións concretas para SQLite ou servizos externos.

Esta separación permite que as vistas se centren fundamentalmente na presentación e na interacción co usuario, mentres que os ViewModels coordinan o estado e as operacións e os repositorios encapsulan o acceso ás fontes de datos. Durante o desenvolvemento tamén se aplicaron criterios de reutilización de widgets, validación de formularios, tratamento de erros, navegación entre pantallas e adaptación da interface a diferentes tamaños.

MARTOLA non pretende constituír nesta primeira versión un produto comercial completo. O proxecto foi abordado de maneira incremental, establecendo primeiro unha versión mínima viable e ampliándoa posteriormente a medida que os módulos principais adquirían estabilidade. Determinadas funcionalidades inicialmente consideradas, como a autenticación de usuarios, a sincronización cloud ou o almacenamento dun histórico meteorolóxico, quedan expresamente reservadas como posibles liñas futuras.

O resultado é unha aplicación funcional que permite aplicar de forma integrada coñecementos relacionados coa programación, persistencia de datos, consumo de APIs, arquitectura software, deseño de interfaces, control de versións e desenvolvemento multiplataforma.

2. Estudo de mercado e alternativas existentes

Antes de definir o alcance definitivo de MARTOLA realizouse unha análise de solucións existentes relacionadas coa planificación e o seguimento de hortas e xardíns. O obxectivo non é demostrar a inexistencia de alternativas, senón identificar enfoques xa consolidados e situar o proxecto dentro dun mercado no que conviven planificadores visuais, diarios de cultivo e asistentes de xardinaría.

Para esta comparación seleccionáronse Planter, From Seed to Spoon, GrowVeg Garden Planner e Gardenize. Trátase de produtos con enfoques parcialmente coincidentes con MARTOLA, pero con diferentes niveis de especialización. A información recolleuse das páxinas e documentación oficiais dispoñibles durante a elaboración da memoria. [1][2][3][4]

2.1. Planter

Planter céntrase especialmente na planificación visual da horta. Permite crear un xardín indicando as súas dimensións, colocar plantas sobre unha cuadrícula e consultar información sobre asociación de cultivos, datas de plantación e variedades. A documentación oficial indica dispoñibilidade en iOS, Android e web, ademais de mecanismos de copia e transferencia de datos entre dispositivos mediante conta. [1]

A súa principal relación con MARTOLA atópase no deseño espacial. Planter ofrece unha planificación máis especializada e orientada a cuadrícula, mentres que o Layout Designer de MARTOLA funciona como unha representación libre e persistente das plantas xa rexistradas nunha horta.

2.2. From Seed to Spoon

From Seed to Spoon presenta un enfoque máis amplo. Integra un planificador visual, datas de plantación personalizadas, información sobre asociación de cultivos e pragas, recordatorios, seguimento mediante notas e fotografías e estimacións de xerminación e colleita. A súa documentación recente tamén describe a creación de camas de cultivo, incorporación de plantas e rexistro de eventos de coidado. [2]

É a alternativa que máis se aproxima á idea de reunir varias necesidades nunha única aplicación. Non obstante, MARTOLA ten un alcance académico máis reducido e pon especial énfase na arquitectura do software, na persistencia local mediante SQLite e na demostración do fluxo completo entre Views, ViewModels, Repositories e fontes de datos.

2.3. GrowVeg Garden Planner

GrowVeg Garden Planner está orientado á planificación detallada do espazo cultivable. A súa guía oficial describe ferramentas para debuxar zonas, engadir plantas e obxectos, calcular cantas plantas caben nun espazo e facilitar tarefas de sucesión de cultivos e rotación. [3]

Fronte a este enfoque especializado na planificación agronómica, MARTOLA utiliza un deseñador máis sinxelo. O seu obxectivo non é calcular densidades ou rotacións, senón ofrecer unha representación visual coherente co resto dos datos almacenados para cada horta.

2.4. Gardenize

Gardenize adopta principalmente o modelo de diario dixital de xardín. Permite organizar plantas e áreas, rexistrar eventos, engadir fotografías e consultar a evolución histórica. A súa oferta inclúe tamén calendario e tarefas, acceso desde móbil e escritorio e funcións adicionais segundo o plan contratado. [4]

Este enfoque é comparable ao seguimento da evolución das plantas de MARTOLA. Gardenize profundiza especialmente no diario fotográfico e no histórico, mentres que MARTOLA combina un seguimento máis básico con meteoroloxía, xeocodificación e deseño espacial.

2.5. Comparativa

Solución	Enfoque principal	Funcionalidades relacionadas con MARTOLA	Diferenza destacable
Planter	Planificación visual	Hortas, plantas, disposición espacial, datas	Planificador de cuadrícula e información de asociación

		de plantación	de cultivos máis especializada.
Seed to Spoon	Asistente integral de cultivo	Planificador, plantas, seguimento, tarefas/recordatorios, localización	Maior catálogo e contido agronómico; incorpora fotos, pragas e recomendacións.
GrowVeg	Planificación do terreo	Deseño espacial, plantas e planificación de cultivos	Maior profundidade no cálculo do espazo, sucesión e rotación de cultivos.
Gardenize	Diario dixital	Plantas, áreas, eventos, evolución e tarefas	Maior énfase no histórico fotográfico, diario e sincronización de información.
MARTOLA	Xestión multiplataforma de hortas	Hortas, plantas, evolución, meteo, tarefas e layout	MVP Local First con SQLite e arquitectura MVVM + Repository desenvolvida como TFC.

2.6. Posicionamento de MARTOLA

A análise mostra que MARTOLA non parte dun problema sen solucións previas. Existen produtos maduros que superan o MVP desenvolvido en número de especies, contido agronómico, sincronización ou planificación avanzada. Por tanto, a diferenciación do proxecto non debe formularse como unha superioridade funcional fronte a estas aplicacións.

O interese de MARTOLA reside na integración, dentro dun proxecto académico controlable, de varias das ideas presentes neste mercado: rexistro estruturado de hortas e plantas, seguimento da evolución, meteoroloxía contextual, tarefas e representación espacial. Todo isto constrúese sobre unha arquitectura propia, con persistencia local e unha base de código multiplataforma.

Esta análise tamén permite identificar oportunidades futuras claras: ampliar o catálogo vexetal, incorporar fotografías e recomendacións, mellorar o planificador, introducir recordatorios avanzados e habilitar sincronización entre dispositivos. Estas posibilidades mantéñense como evolución futura e non forman parte dos requisitos do MVP actual.

3. Obxectivos e alcance

3.1. Obxectivo xeral

O obxectivo principal do proxecto é deseñar e desenvolver unha aplicación multiplataforma que permita centralizar a xestión básica de hortas e xardíns, proporcionando ao usuario ferramentas para organizar os espazos cultivados, as plantas que conteñen, a súa evolución, a súa distribución e determinadas tarefas relacionadas co seu mantemento.

Paralelamente ao obxectivo funcional, o proxecto pretende construír a aplicación sobre unha arquitectura organizada e extensible, evitando concentrar a lóxica de negocio, a interface e o acceso aos datos nunha mesma capa.

3.2. Obxectivos específicos

1. Desenvolver unha aplicación cunha única base de código capaz de funcionar en diferentes plataformas mediante Flutter.
2. Permitir crear, consultar, modificar e eliminar hortas ou xardíns.
3. Almacenar para cada horta información relevante como o nome, localización, superficie e coordenadas xeográficas.
4. Permitir asociar diferentes plantas a unha horta e xestionar os seus datos principais.
5. Dispoñer dun catálogo local de especies que permita relacionar cada planta concreta cunha especie.
6. Permitir realizar un seguimento básico da evolución das plantas mediante rexistros asociados a unha data e, opcionalmente, unha altura e anotacións.
7. Integrar un servizo externo de xeocodificación que permita obter as coordenadas necesarias a partir da localización dunha horta.
8. Integrar información meteorolóxica actual mediante OpenWeather utilizando as coordenadas asociadas á horta.
9. Permitir representar visualmente a disposición das plantas mediante un deseñador no que os elementos poidan ser incorporados, desprazados e retirados, mantendo a posición establecida.
10. Incorporar unha xestión básica de tarefas que permita rexistrar actividades pendentes e controlar o seu estado.
11. Manter os datos propios da aplicación mediante persistencia local con SQLite.
12. Garantir a integridade das principais relacións entre entidades mediante claves foráneas e comportamentos apropiados ante a eliminación de datos relacionados.
13. Organizar o código seguindo unha arquitectura MVVM simplificada e o Repository Pattern.
14. Xestionar o estado da aplicación mediante Provider e ChangeNotifier.
15. Deseñar unha interface reutilizable e adaptable aos tamaños de pantalla correspondentes ás plataformas obxectivo.
16. Utilizar Git e GitHub para manter o control de versións e a documentación asociada ao desenvolvemento.

3.3. Alcance da versión desenvolvida

A versión actual de MARTOLA céntrase nun modelo de funcionamento principalmente local. SQLite actúa como mecanismo de persistencia para os datos internos, mentres que os servizos meteorolóxicos son obtidos baixo demanda desde unha API externa.

O esquema da base de datos evolucionou durante o desenvolvemento ata a versión 6. Entre as entidades implementadas atópanse hortas, especies, plantas, rexistros de evolución, elementos do deseño visual e tarefas.

O alcance actual non inclúe un sistema de autenticación. Polo tanto, a aplicación non diferencia aínda entre diferentes contas de usuario. Tampouco existe sincronización remota dos datos entre dispositivos.

A meteoroloxía está integrada como información de consulta, pero non se almacena un histórico meteorolóxico en SQLite. Os datos recibidos desde OpenWeather mantéñense no estado da aplicación sen constituír unha entidade persistente.

Esta delimitación responde á decisión de priorizar unha aplicación funcional e arquitectonicamente organizada fronte á incorporación dun número excesivo de características cun menor nivel de desenvolvemento.

4. Análise de requisitos

4.1. Requisitos funcionais

Código	Requisito	Descrición
RF01	Xestión de hortas	O sistema debe permitir crear, visualizar, modificar e eliminar hortas.
RF02	Información xeográfica da horta	O sistema debe permitir asociar unha localización e coordenadas xeográficas a unha horta.
RF03	Xestión de plantas	O usuario debe poder engadir plantas a unha horta, consultar os seus datos, modificalas e eliminalas.
RF04	Asociación entre plantas e especies	Cada planta rexistrada debe estar asociada a unha especie existente no catálogo.
RF05	Catálogo de especies	A aplicación debe proporcionar un catálogo local básico de especies para o rexistro e modificación das plantas.
RF06	Seguimento da evolución	O sistema debe permitir engadir, consultar, modificar e eliminar rexistros de evolución asociados a unha planta.
RF07	Datos opcionais de evolución	Un rexistro debe poder almacenar a data e, opcionalmente, a altura da planta e notas adicionais.

RF08	Consulta meteorolóxica	O usuario debe poder consultar as condicións meteorolóxicas actuais asociadas á localización dunha horta.
RF09	Xeocodificación	A aplicación debe poder consultar un servizo externo para obter posibles localizacións e coordenadas xeográficas.
RF10	Deseño da horta	O usuario debe poder incorporar ao deseño visual as plantas existentes nunha horta.
RF11	Reorganización visual	As plantas incorporadas ao deseño deben poder ser desprazadas dentro da superficie dispoñible e conservar a súa posición.
RF12	Retirada do deseño	O usuario debe poder retirar unha planta do deseño sen eliminar a planta da horta.
RF13	Xestión de tarefas	A aplicación debe permitir crear e xestionar tarefas e distinguir o seu estado de realización.
RF14	Persistencia	Os cambios realizados sobre os datos propios da aplicación deben conservarse despois de pechar e volver iniciar MARTOLA.
RF15	Navegación	O usuario debe poder navegar entre o panel principal e os diferentes módulos e regresar ás pantallas anteriores mantendo un fluxo coherente.

4.2. Requisitos non funcionais

Código	Requisito	Descrición
RNF01	Multiplataforma	A aplicación debe utilizar unha base de código común que permita a súa execución nas plataformas definidas

		como obxectivo.
RNF02	Persistencia local	A información interna da aplicación debe almacenarse localmente mediante SQLite.
RNF03	Separación de responsabilidades	A interface, a xestión do estado e o acceso aos datos deben permanecer separados mediante a arquitectura definida para o proxecto.
RNF04	Integridade dos datos	As relacións existentes entre as entidades almacenadas deben manter a súa coherencia mediante claves foráneas e restricións adecuadas.
RNF05	Usabilidade	As operacións principais deben poder realizarse mediante fluxos de navegación sinxelos e formularios con validación.
RNF06	Adaptabilidade	A interface debe poder reorganizar o seu contido segundo o espazo dispoñible, evitando depender das dimensións dun único dispositivo.
RNF07	Tratamento de erros externos	Os servizos de rede deben contemplar tempos de espera, problemas de conexión, erros do servidor ou credenciais non válidas.
RNF08	Mantemento	O código debe organizarse por responsabilidades e funcionalidades para facilitar a localización e modificación dos diferentes módulos.
RNF09	Seguridade básica da configuración	As credenciais para servizos externos non deben quedar almacenadas directamente no código fonte nin publicarse no repositorio.
RNF10	Control de versións	O desenvolvemento debe quedar rexistrado mediante Git, permitindo conservar a evolución incremental do proxecto.

4.3. Casos de uso principais

MARTOLA non incorpora autenticación na versión actual, polo que os casos de uso parten dun único actor xenérico: o usuario da aplicación. Os casos principais cubertos polo MVP son os seguintes:

- CU01 — Xestionar hortas: crear, consultar, editar e eliminar hortas ou xardíns.
- CU02 — Localizar unha horta: buscar unha localización, seleccionar un resultado e gardar as súas coordenadas.
- CU03 — Consultar meteoroloxía: obter o tempo actual para unha horta que dispoña de coordenadas.
- CU04 — Xestionar plantas: engadir, consultar, editar e eliminar plantas asociadas a unha horta.
- CU05 — Rexistrar evolución: crear consultar, editar e eliminar rexistros cronolóxicos con data, altura e notas.
- CU06 — Deseñar a disposición: engadir plantas ao taboleiro, movelas, retirar elementos e conservar a posición.
- CU07 — Xestionar tarefas: crear, consultar, completar e eliminar tarefas básicas.
- CU08 — Consultar o Dashboard: visualizar accesos rápidos e contadores procedentes do estado da aplicación.

5. Planificación e metodoloxía

5.1. Enfoque de desenvolvemento

O desenvolvemento de MARTOLA realizouse de maneira incremental. O proxecto partiu dunha definición inicial do problema, dos requisitos e dun MVP, e foi incorporando módulos funcionais de forma progresiva. Este enfoque permitiu validar cada bloque antes de aumentar a complexidade e adaptar as prioridades segundo o estado real da aplicación.

A planificación non se empregou como unha secuencia ríxida e inmutable. O roadmap foi actualizándose a medida que se completaban funcionalidades e aparecían novas necesidades técnicas. Deste xeito, elementos que inicialmente se consideraban ampliacións, como a meteoroloxía asociada á localización dunha horta ou o deseñador visual, acabaron integrándose no alcance funcional final, mentres que outras ideas, como a autenticación, a sincronización cloud ou o histórico meteorolóxico persistente, mantivéronse como liñas futuras.

5.2. Desenvolvemento orientado ao MVP

A primeira definición do produto mínimo viable centrouse na navegación funcional, a xestión de estado, a persistencia SQLite, a xestión de hortas e plantas e o rexistro da evolución. Unha vez estabilizado ese núcleo, o proxecto ampliouse coa integración meteorolóxica, a xeocodificación, o Layout Designer e, finalmente, un módulo básico de tarefas.

Esta priorización permitiu que cada ampliación se apoiase nunha arquitectura xa funcional. Os novos módulos reutilizaron os mesmos principios de separación entre vistas, ViewModels e repositorios, evitando crear solucións independentes para cada funcionalidade.

5.3. Fases principais

O traballo pode resumirse nas seguintes fases: análise e planificación inicial; deseño do modelo de datos e da navegación; aprendizaxe e implementación da base de Flutter; configuración da arquitectura e Provider; persistencia SQLite; CRUD de hortas; xestión de plantas e especies; rexistros de evolución; integración REST con OpenWeather e xecodificación; deseñador visual persistente; módulo básico de tarefas; adaptación e revisión da interface; probas manuais, análise estática, preparación de builds e documentación final.

O desenvolvemento do Layout Designer é un exemplo da evolución incremental seguida. A funcionalidade incorporou posicionamento mediante coordenadas normalizadas, movemento por arrastre, límites do taboleiro, prevención de solapamentos e persistencia das posicións. Unha vez comprobada a súa estabilidade, as melloras adicionais de interacción quedaron como refinamentos opcionais en lugar de bloquear o avance do proxecto.

5.4. Aprendizaxe aplicada

O proxecto tamén se utilizou como proceso de aprendizaxe progresiva de Flutter e Dart. As sesións de desenvolvemento asociaron conceptos concretos cun resultado funcional dentro de MARTOLA: composición de widgets, navegación, formularios, Provider e ChangeNotifier, modelos, SQLite, Repository Pattern, consumo de APIs REST, asíncronía e deseño responsive.

Esta metodoloxía permitiu introducir conceptos novos no momento no que eran necesarios para resolver unha necesidade real da aplicación. Como consecuencia, a documentación de desenvolvemento non só recolle o resultado final, senón tamén a evolución das decisións e das competencias adquiridas.

5.5. Control de versións e documentación

O código fonte mantívose baixo control de versións con Git e un repositorio remoto en GitHub. Os cambios relevantes foron incorporándose mediante commits ao finalizar bloques de traballo, permitindo conservar a evolución do proxecto.

Paralelamente mantivéronse documentos específicos para o contexto do proxecto, arquitectura, base de datos, interface, roadmap, guía de desenvolvemento e notas de aprendizaxe. Esta documentación serviu como apoio durante a implementación e constitúe a base para a elaboración da presente memoria. No peche da versión 1.0.0 actualizouse tamén o README e organizáronse os diagramas finais, mantendo os artefactos binarios e o vídeo de demostración na Release en lugar de incorporalos como ficheiros versionados.

6. Tecnoloxías e ferramentas

6.1. Flutter e Dart

Flutter constitúe o framework principal da aplicación e Dart a linguaxe de programación empregada. A elección responde ao obxectivo de manter unha única base de código para Android e escritorio, reducindo a duplicación entre plataformas. Flutter proporciona ademais un sistema de widgets adecuado para construír unha interface adaptable e reutilizable.

6.2. Provider e ChangeNotifier

Provider utilízase para proporcionar e compartir os ViewModels na árbore de widgets. Os ViewModels estenden ChangeNotifier e notifican ás vistas cando cambia o estado que deben representar. Esta solución permite manter a lóxica de estado fóra das pantallas e encaixa co MVVM simplificado adoptado no proxecto.

6.3. SQLite e librarías de persistencia

SQLite é o mecanismo de persistencia local de MARTOLA. Para Android emprégase sqflite e, nos sistemas de escritorio soportados, sqflite_common_ffi permite utilizar SQLite mediante FFI. DatabaseService centraliza a apertura e evolución do esquema. A base de datos inclúe migracións entre versións e activa as claves foráneas para manter a integridade referencial.

A librería path emprégase para construír rutas de maneira independente do sistema operativo e path_provider permite obter directorios apropiados da aplicación. A base de datos martola.db gárdase no directorio de soporte da aplicación, evitando utilizar o directorio persoal de documentos do usuario para datos internos.

6.4. HTTP, JSON e OpenWeather

A integración de datos externos realízase mediante o paquete http. Os servizos constrúen peticións HTTPS, procesan respostas JSON e transforman os datos recibidos en modelos do dominio. OpenWeather actúa como provedor da meteoroloxía actual e da xeocodificación empregada para resolver localizacións.

A integración contempla códigos de estado HTTP, tempos máximos de espera e excepcións específicas de servizo. A clave da API non se almacena directamente no código fonte: introdúcese como valor de compilación mediante String.fromEnvironment e mantense fóra do control de versións.

6.5. Git e GitHub

Git utilízase para o control de versións e GitHub como repositorio remoto. Ademais do código fonte, o repositorio conserva a documentación técnica e os diagramas do proxecto. Para a versión

1.0.0 preparouse tamén unha GitHub Release con artefactos distribuíbles para Windows x64 e Android, acompañados dun vídeo de demostración do funcionamento da aplicación.

6.6. VS Code e ferramentas de deseño

Visual Studio Code foi o contorno principal de desenvolvemento. Para o traballo visual empregouse Figma como ferramenta de referencia para wireframes e organización do sistema de deseño, mentres que a documentación técnica inclúe diagramas e representacións do modelo e da arquitectura.

6.7. Criterio de selección

As tecnoloxías escollidas procuran manter un equilibrio entre o alcance académico, a aprendizaxe e a posibilidade de evolución futura. Flutter facilita a orientación multiplataforma; SQLite ofrece persistencia local sen infraestrutura adicional; Provider resolve a xestión de estado cunha complexidade moderada; e o Repository Pattern permite que as fontes de datos concretas non condicionen directamente a interface.

7. Arquitectura da aplicación

7.1. Visión xeral

MARTOLA utiliza unha arquitectura MVVM simplificada combinada co Repository Pattern e cun enfoque local first. O fluxo principal de dependencias pode representarse como View → ViewModel → Repository → DatabaseService ou servizo externo → SQLite ou API.

A finalidade desta estrutura é separar a presentación, o estado da aplicación e o acceso aos datos. Esta separación facilita o mantemento e permite incorporar novas implementacións de persistencia ou servizos externos sen trasladar esa complexidade ás pantallas.

A Figura 1 resume a composición arquitectónica final e a separación entre persistencia local e servizos externos.



Figura 1. Esquema da arquitectura final de MARTOLA.

7.2. Capa de modelos

Os modelos representan as entidades do dominio, entre elas Garden, PlantSpecies, GardenPlant, PlantEvolutionRecord, GardenLayoutItem e as tarefas, ademais dos modelos empregados para

información externa como WeatherData e os resultados de xeocodificación. Os modelos non conteñen widgets, navegación nin acceso directo a SQLite ou HTTP.

Para as entidades persistentes utilízanse conversións entre os tipos do dominio e a representación almacenada en SQLite. As datas trabállanse como DateTime no dominio e convértense á representación persistente correspondente.

7.3. Views

As Views están formadas polas pantallas e widgets de Flutter. A súa responsabilidade principal é presentar o estado, recoller interaccións do usuario, validar formularios e iniciar accións a través dos ViewModels. Non realizan consultas SQL nin chamadas HTTP directas.

A navegación entre pantallas utiliza os identificadores ou obxectos necesarios segundo o caso, mentres que o estado compartido se obtén mediante Provider. Widgets reutilizables permiten evitar duplicación de estruturas visuais comúns.

7.4. ViewModels

Os ViewModels actúan como intermediarios entre a interface e os repositorios. Encapsulan o estado observable, executan operacións asíncronas e notifican os cambios mediante ChangeNotifier. Entre os ViewModels integrados atópanse os correspondentes a hortas, especies, plantas, evolución, meteoroloxía, xeocodificación, deseño da horta e tarefas.

Esta capa tamén permite representar estados propios das operacións remotas, como carga, resultado e erro, sen trasladar á View os detalles do acceso á API.

7.5. Repositories

Os repositorios definen contratos para o acceso aos datos. As Views e ViewModels non dependen da implementación concreta de SQLite ou OpenWeather, senón das interfaces correspondentes. Para a persistencia local existen implementacións SQLite específicas para cada módulo; para os datos meteorolóxicos e de xeocodificación utilízanse implementacións apoiadas nos servizos de OpenWeather.

Este desacoplamento permite substituír ou ampliar unha fonte de datos no futuro con menor impacto sobre as capas superiores. Por exemplo, podería incorporarse outro provedor meteorolóxico mantendo o contrato empregado polo ViewModel.

7.6. Services e fontes de datos

DatabaseService centraliza a creación e apertura da base de datos, a activación das claves foráneas, as migracións do esquema e os datos iniciais do catálogo de especies. Os servizos WeatherService e GeocodingService encapsulan as peticións HTTP e o tratamento dos erros propios da comunicación externa.

7.7. Composición e inxección de dependencias

A composición principal realízase en `main.dart`. Nese punto créanse os servizos e repositorios concretos e entréganse á aplicación. `MultiProvider` crea os `ViewModels` coas dependencias correspondentes e os fai dispoñibles para as `Views`.

Esta inxección mediante construtores evita que os `ViewModels` creen internamente as súas fontes de datos e fai máis explícitas as dependencias entre capas.

7.8. Fluxos representativos

Unha operación local segue, de forma xeral, o fluxo `View` → `ViewModel` → `Repository` → `DatabaseService` → `SQLite`. Unha consulta meteorolóxica segue `View` → `WeatherViewModel` → `WeatherRepository` → `WeatherService` → `OpenWeather API`. A xeocodificación segue un fluxo equivalente a través do seu propio repositorio e servizo.

No caso do `Layout Designer`, a mesma arquitectura úsase para unha interacción máis dinámica: a posición móstrase e actualízase localmente durante o arrastre e persístese ao finalizar a interacción. Isto evita escribir continuamente na base de datos durante cada pequeno movemento.

7.9. Boas prácticas aplicadas

Entre as prácticas empregadas destacan a separación de responsabilidades, o uso de contratos de repositorio, a inxección de dependencias, a reutilización de widgets, a validación dos formularios, o uso de `async/await` para operacións asíncronas, a comprobación de `mounted` tras operacións que poden completar despois dun cambio de pantalla, o tratamento de excepcións de servizos externos e a exclusión das credenciais do repositorio.

A arquitectura non pretende reproducir unha implementación completa e ríxida de MVVM, senón aplicar os seus principios de maneira proporcionada ao tamaño e aos obxectivos dun TFC. Por este motivo descríbese ao longo da documentación como MVVM simplificado.

8. Deseño e persistencia dos datos

8.1. Estratexia de persistencia

MARTOLA adopta unha estratexia local first: a información propia do usuario persístese nunha base de datos `SQLite` denominada `martola.db`. A apertura, configuración, creación e evolución do esquema centralízanse en `DatabaseService`. O ficheiro gárdase no directorio de soporte propio da aplicación obtido mediante `getApplicationSupportDirectory()`, evitando empregar a carpeta persoal de documentos do usuario para datos internos. Deste modo, os repositorios non necesitan coñecer detalles sobre a localización física do ficheiro nin repetir a lóxica de inicialización da base de datos.

A versión final do esquema é a 6. A evolución realizouse de maneira incremental mediante migracións acumulativas, de forma que unha instalación creada cunha versión anterior pode aplicar sucesivamente os cambios necesarios sen recrear toda a base de datos nin perder os datos xa almacenados.

8.2. Modelo relacional implementado

O esquema físico final está formado por seis táboas: gardens, plant_species, garden_plants, plant_evolution_records, garden_layout_items e tasks. As relacións principais permiten representar unha horta con varias plantas, cada planta asociada a unha especie, varios rexistros de evolución por planta e, opcionalmente, unha posición no deseño visual. As tarefas constitúen nesta versión un módulo independente. O deseño inicial era máis amplo e contemplaba, entre outros elementos, usuarios, histórico meteorolóxico persistente e máis atributos para especies e layout. Durante a implementación o modelo reduciuse conscientemente para priorizar un MVP completo e estable; os diagramas iniciais consérvanse no repositorio como evidencia desta evolución.

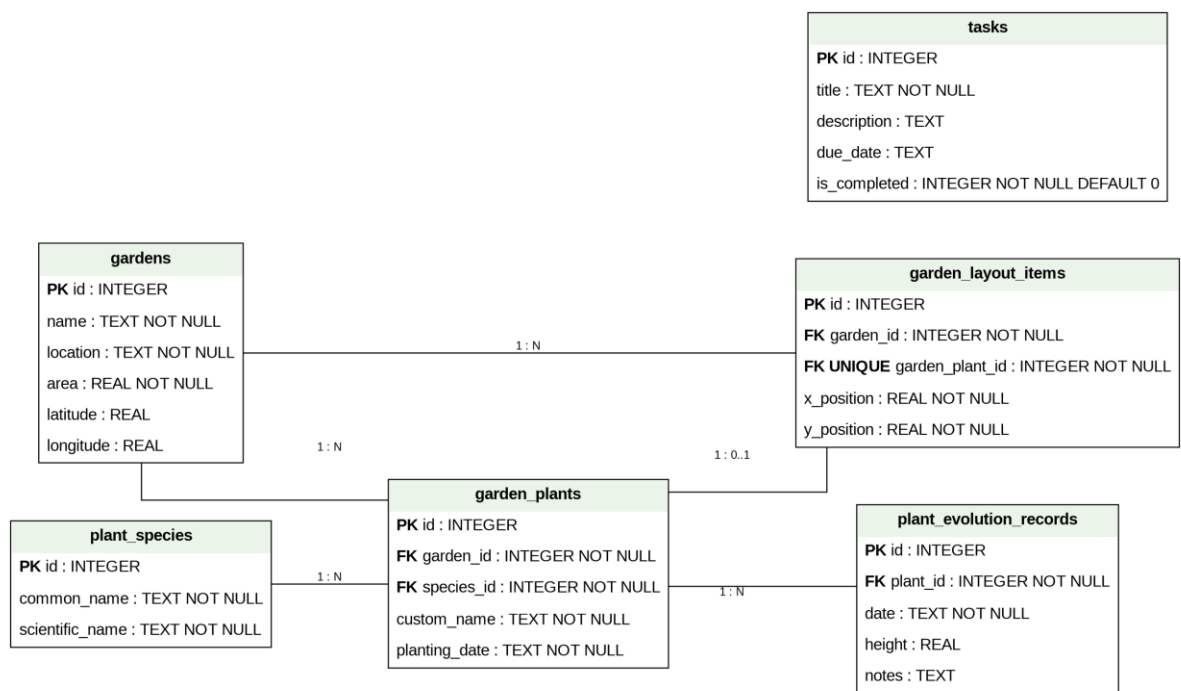


Figura 2. Diagrama entidade-relación do esquema físico final, versión 6.

Táboa	Finalidade
gardens	Hortas ou xardíns xestionados polo usuario.
plant_species	Catálogo local de especies.
garden_plants	Plantas concretas asociadas a unha horta e unha especie.
plant_evolution_records	Rexistros temporais da evolución dunha

	planta.
garden_layout_items	Posición persistente dunha planta no deseñador visual.
tasks	Tarefas básicas pendentes ou completadas.

8.3. Táboa gardens

A táboa gardens almacena os datos principais de cada horta: identificador, nome, localización e superficie. Posteriormente incorporáronse latitude e longitude para poder asociar a horta a consultas meteorolóxicas sen repetir a xeocodificación en cada petición.

Campo	Tipo	Restrición / finalidade
id	INTEGER	Clave primaria autoincremental.
name	TEXT	Nome obrigatorio.
location	TEXT	Localización obrigatoria.
area	REAL	Superficie obrigatoria.
latitude	REAL	Coordenada opcional.
longitude	REAL	Coordenada opcional.

8.4. Especies e plantas

plant_species contén un catálogo local deliberadamente sinxelo, composto polo nome común e o nome científico. Cando a base de datos está baleira, DatabaseService introduce un pequeno conxunto inicial de especies para que o módulo de plantas poida utilizarse desde a primeira execución. Durante a revisión final detectouse unha duplicación do catálogo causada por aperturas concorrentes da base de datos antes de quedar resolta a instancia compartida. A inicialización foi axustada para que as chamadas simultáneas reutilicen o mesmo proceso de apertura; unha vez aplicada a corrección, o seeding volveu executarse unha única vez sobre unha base baleira.

garden_plants representa cada exemplar ou cultivo concreto. garden_id establece a relación coa horta e species_id coa especie. O nome personalizado e a data de plantación completan a información básica. A data almacénase como texto en formato ISO 8601, realizando a conversión na fronteira coa persistencia.

8.5. Evolución das plantas

plant_evolution_records permite manter un historial asociado a unha planta. A data é obrigatoria, mentres que a altura e as notas son opcionais. Esta decisión permite rexistrar unha observación aínda que o usuario non dispoña dunha medición numérica.

8.6. Persistencia do Layout Designer

`garden_layout_items` garda a relación entre unha horta, unha planta e a súa posición visual. As coordenadas `x_position` e `y_position` almacénanse normalizadas, é dicir, como valores relativos ao tamaño do taboleiro. Isto evita ligar a disposición a unha resolución concreta e permite reconstruíla en superficies de distinto tamaño.

`garden_plant_id` é único, polo que unha mesma planta non pode aparecer dúas veces no deseño. Tanto a relación coa horta como a relación coa planta empregan `ON DELETE CASCADE`, facendo que os elementos visuais deixen de existir automaticamente cando se elimina a entidade da que dependen.

8.7. Tarefas

A versión 6 engadiu a táboa `tasks` para soportar un módulo de tarefas deliberadamente básico. Cada rexistro contén título, descrición opcional, data límite opcional e un indicador de finalización almacenado como `INTEGER`. Neste MVP as tarefas non teñen clave foránea cara a unha horta ou planta, o que reduce a complexidade do módulo e deixa esa asociación como posible ampliación.

8.8. Integridade referencial

SQLite configúrase con `PRAGMA foreign_keys = ON`. A eliminación dunha horta propágase ás súas plantas e aos elementos do deseño mediante `ON DELETE CASCADE`; a eliminación dunha planta propágase aos seus rexistros de evolución e ao seu elemento de layout. Pola contra, a relación entre `plant_species` e `garden_plants` utiliza `ON DELETE RESTRICT` para impedir eliminar unha especie mentres existan plantas que dependan dela.

8.9. Migracións do esquema

Versión	Cambio principal
v1 → v2	Incorporación de especies e plantas e primeiras relacións mediante claves foráneas.
v2 → v3	Incorporación dos rexistros de evolución.
v3 → v4	Engadido de latitude e longitude ás hortas.
v4 → v5	Incorporación de <code>garden_layout_items</code> .
v5 → v6	Incorporación do módulo de tarefas.

As migracións están implementadas mediante comprobacións acumulativas sobre `oldVersion`. Este enfoque permite que unha instalación antiga execute máis dunha migración nunha única apertura se é necesario.

8.10. Datos non persistentes e alcance futuro

Os datos meteorolóxicos actuais non se gardan en SQLite. `WeatherData` representa a resposta obtida desde OpenWeather e mantense no estado da aplicación durante a súa utilización. A táboa

weather_records, igual que a entidade de usuarios, queda fóra do esquema físico final e resérvase para unha posible evolución do proxecto.

9. Deseño da interface e navegación

9.1. Criterios xerais de interface

A interface de MARTOLA constrúese con Material e procura manter unha estrutura visual sinxela e consistente. O tema global defínese desde MaterialApp mediante un ColorScheme xerado a partir dunha cor verde, acorde coa temática da aplicación. Os espazados, Cards, botóns e formularios reutilizan patróns comúns para reducir diferenzas innecesarias entre pantallas.

9.2. Navegación principal

HomeScreen funciona como punto de entrada e conduce ao Dashboard. O panel principal actúa como concentrador das funcionalidades e ofrece acceso ás hortas, ás tarefas e ás accións rápidas. Desde o listado de hortas accédese ao detalle de cada horta e, a partir deste, ás plantas, á meteoroloxía asociada e ao deseñador visual. Os detalles das plantas permiten acceder ao seu historial de evolución.

9.3. Widgets reutilizables

Durante o desenvolvemento separáronse elementos repetibles en widgets propios. No Dashboard utilízanse Cards especializadas para resumir hortas, tarefas e accións rápidas, mentres que outros módulos empregan compoñentes reutilizables para presentar información. Esta composición mantén as pantallas principais máis lexibles e facilita cambios visuais localizados.

9.4. Formularios e interacción

As operacións de creación e edición realízanse mediante formularios con controladores e validación. A navegación mediante Navigator e MaterialPageRoute permite abrir pantallas de creación, detalle ou edición e regresar ao contexto anterior unha vez completada a operación. Para accións destrutivas ou sensibles utilízanse diálogos de confirmación cando resulta apropiado.

9.5. Adaptación a diferentes tamaños

A interface foi revisada manualmente con diferentes anchos de dispositivo e tamaños de ventá, incluíndo móbil, tablet e escritorio. O uso de Expanded, SingleChildScrollView, LayoutBuilder e estruturas flexibles permite aproveitar o espazo dispoñible. Detectouse un overflow en

determinadas filas do Dashboard con anchos extremadamente reducidos, aproximadamente por baixo de 314 píxeles lóxicos; ao quedar fóra do rango práctico previsto para os dispositivos obxectivo, decidiuse non aumentar a complexidade do MVP para resolver ese caso límite.

9.6. Evidencias visuais da interface

As seguintes capturas corresponden á versión final revisada e mostran os fluxos principais e a adaptación da interface a diferentes tamaños.

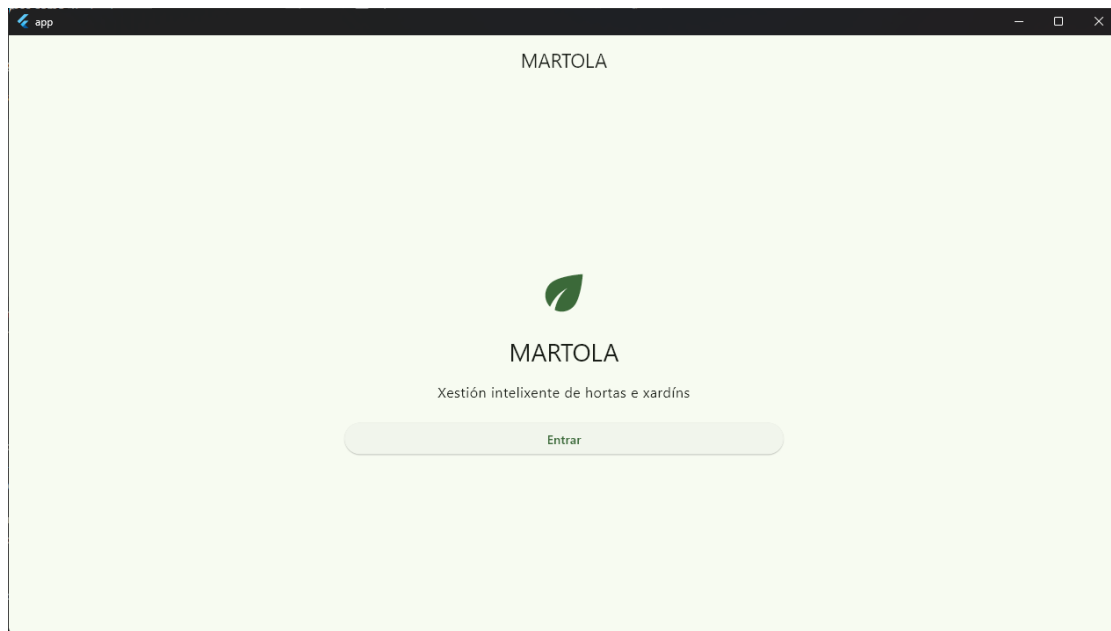


Figura 3. Pantalla de inicio de MARTOLA.

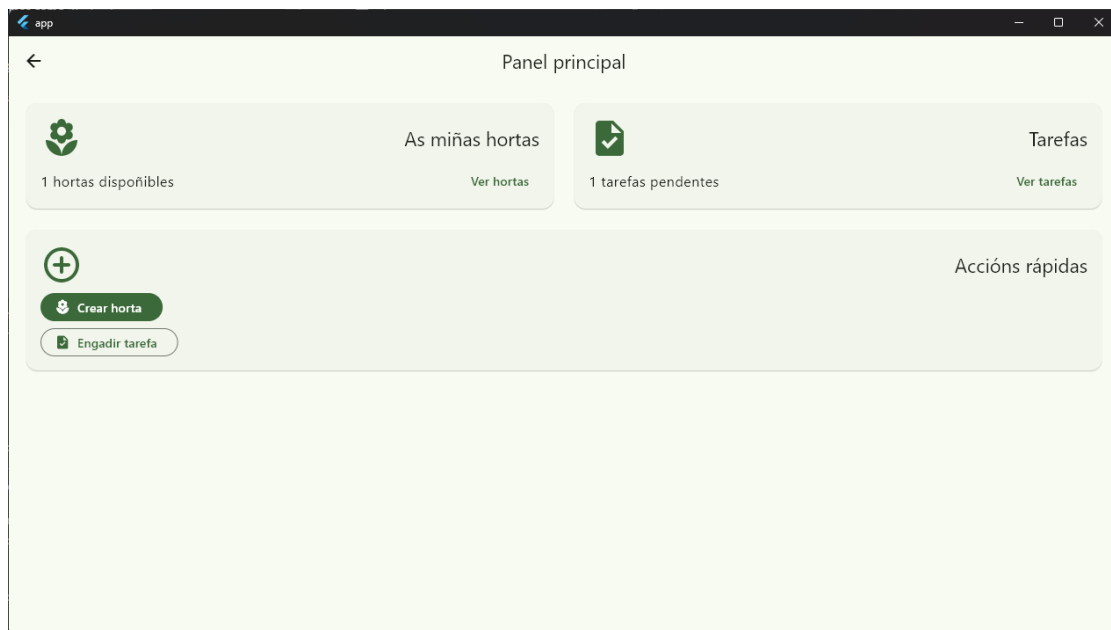


Figura 4. Dashboard e acceso ás funcionalidades principais.

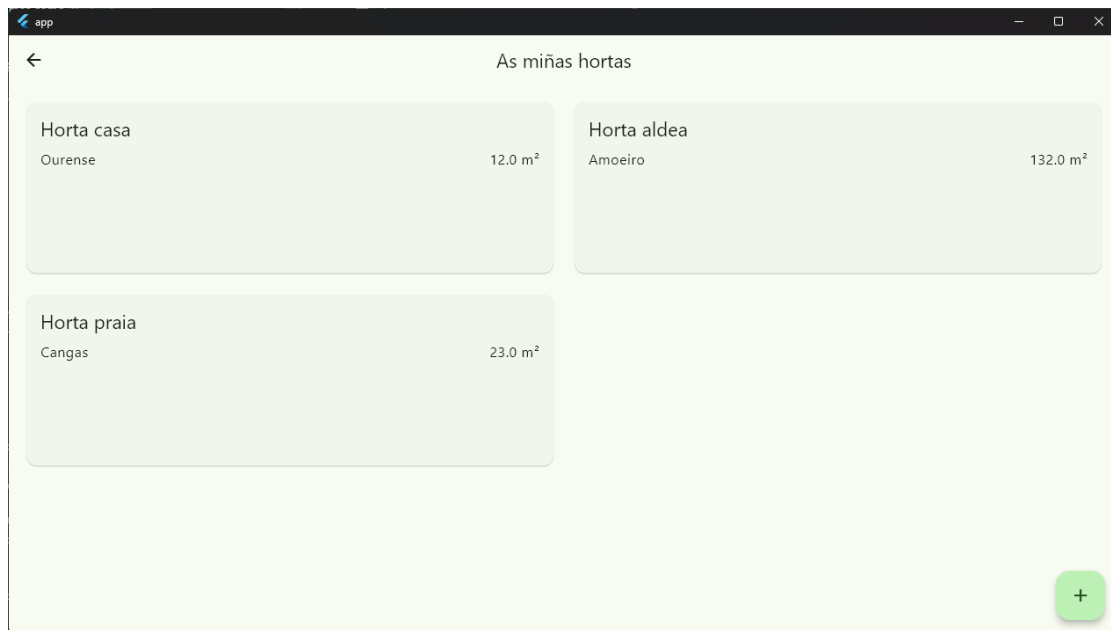


Figura 5. Listado de hortas.

The screenshot shows a mobile application window titled 'Formulario nova horta'. It has a light green background and a dark header bar with a back arrow and the title. The form contains three input fields. The first field is labeled 'Nome' and contains the text 'Horta aldea'. The second field is labeled 'Localización' and contains the text 'Amoeiro'. To the right of this field is a button labeled 'Buscar localización'. Below the location field, there are two radio button options, both labeled 'Amoeiro — Galicia, ES'. The first option has coordinates '42.4142102, -7.9486807'. The second option has coordinates '42.40829895, -7.959688422619871'. The third field is labeled 'Superficie (m²)' and contains the text '132'. At the bottom of the form, there is a green button labeled 'Crear horta'.

Figura 6. Formulario de creación dunha horta.

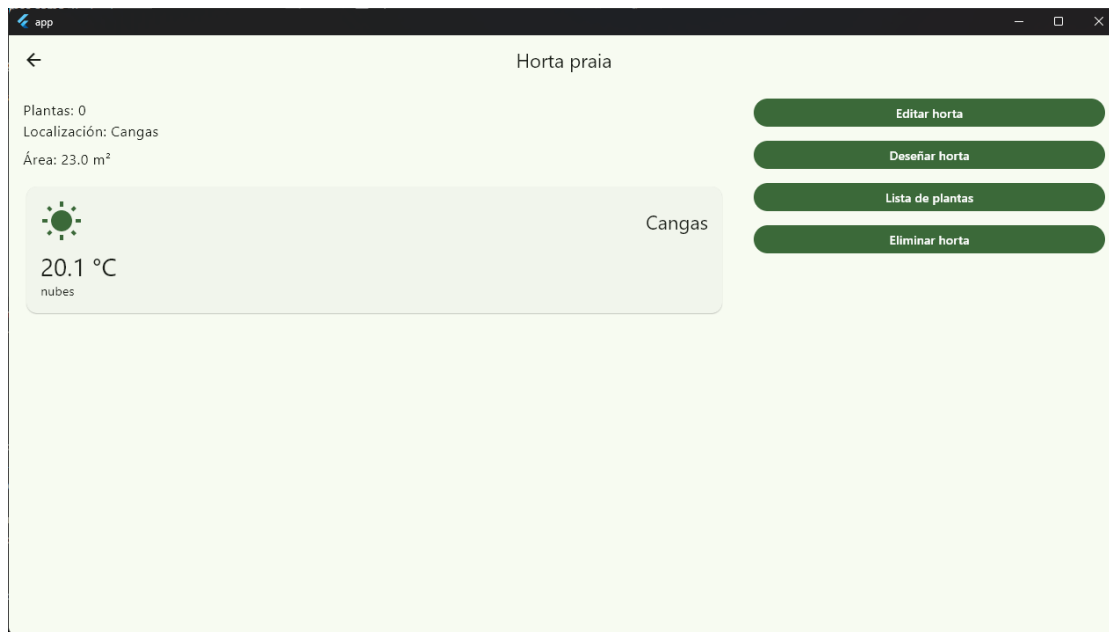


Figura 7. Detalle dunha horta e información contextual.

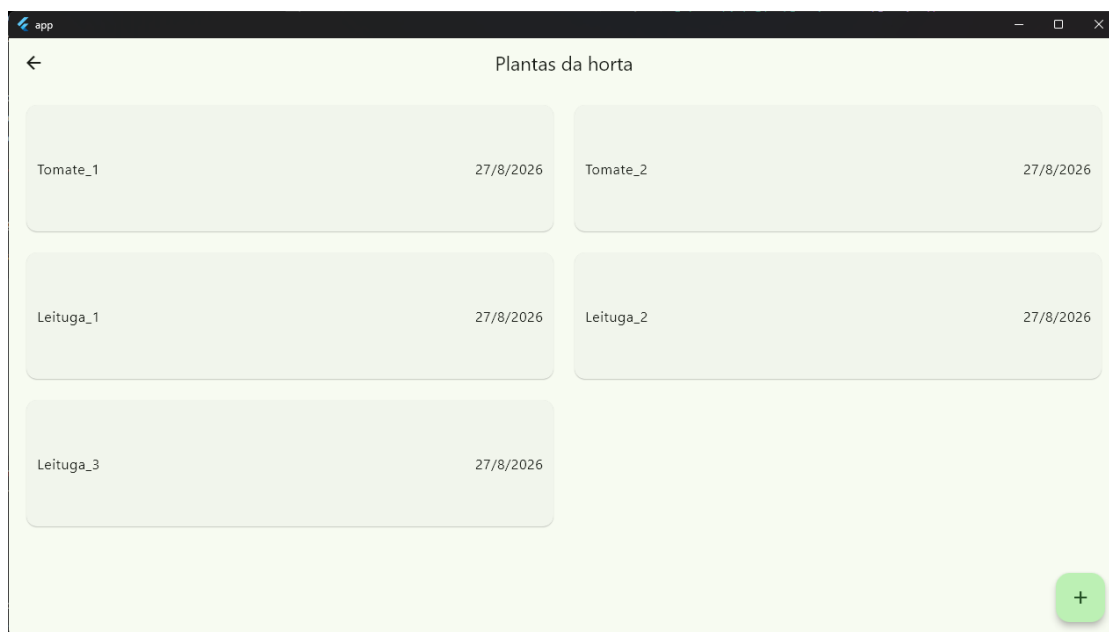


Figura 8. Xestión de plantas asociadas a unha horta.



Figura 9. Seguimento da evolución dunha planta.

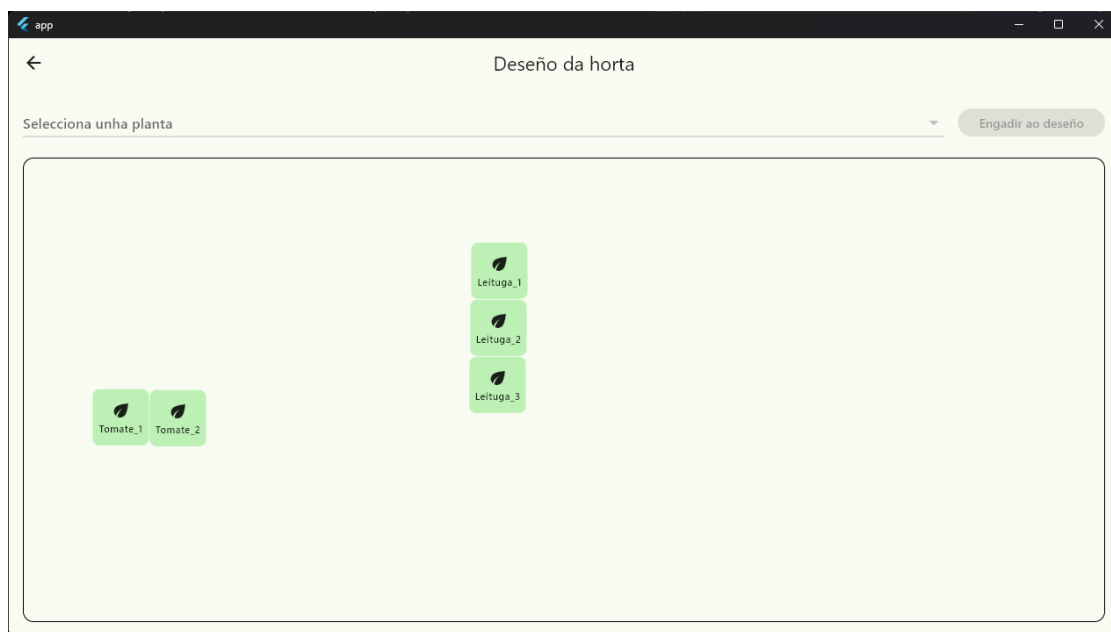


Figura 10. Layout Designer con disposición persistente das plantas.

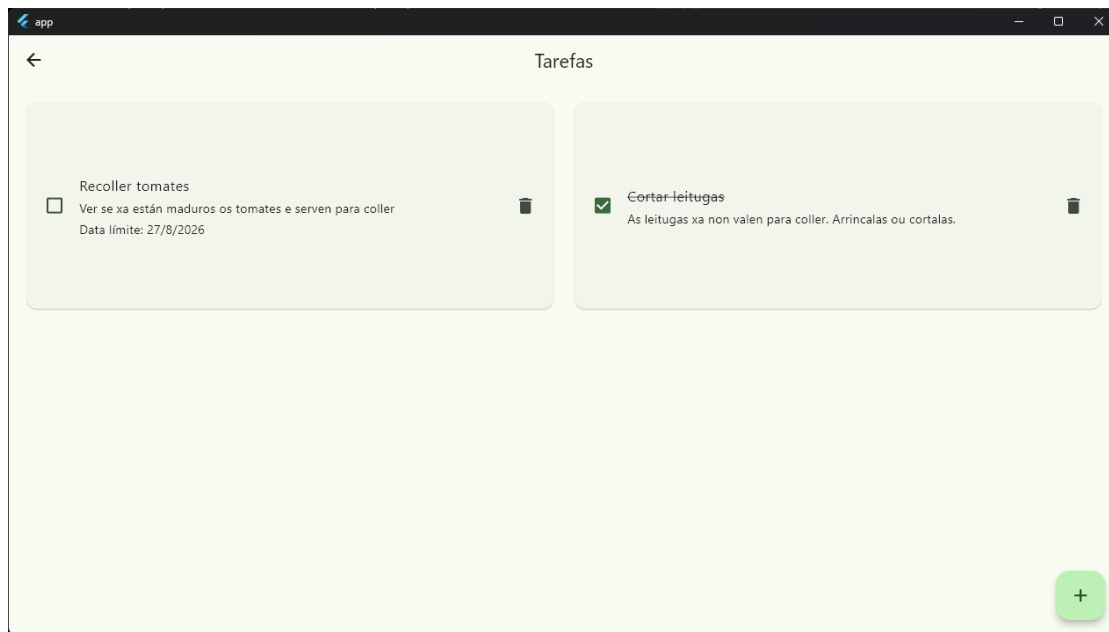


Figura 11. Módulo básico de tarefas.



Figura 12. Exemplo de adaptación da interface a un ancho reducido.

10. Implementación das funcionalidades

10.1. Home e Dashboard

HomeScreen presenta a identidade básica de MARTOLA e un acceso directo ao panel principal. O Dashboard resume información relevante e centraliza a navegación mediante Cards. O contador de hortas e o de tarefas proceden dos ViewModels correspondentes, polo que a interface se actualiza cando cambia o estado persistido.

10.2. Xestión de hortas

O módulo de hortas implementa un CRUD persistente completo. O usuario pode crear, listar, consultar, editar e eliminar hortas. Cada horta conserva nome, localización, superficie e, cando están dispoñibles, coordenadas. GardensViewModel actúa como intermediario entre as vistas e GardenRepository, cuxa implementación SQLite realiza as operacións sobre a base de datos.

10.3. Xeocodificación e meteoroloxía

A localización dunha horta pode resolverse mediante o servizo de xeocodificación de OpenWeather. GeocodingService realiza a petición HTTP e transforma os resultados externos en modelos propios. Unha vez seleccionadas as coordenadas, estas quedan persistidas coa horta.

A consulta meteorolóxica reutiliza latitude e longitude e percorre o fluxo WeatherViewModel → WeatherRepository → OpenWeatherRepository → WeatherService → OpenWeather API. O servizo emprega unidades métricas e solicita a resposta en galego. Tamén contempla timeout, erros de conexión, credenciais non válidas e erros do servidor mediante excepcións específicas.

10.4. Plantas e catálogo de especies

Cada horta pode conter varias GardenPlant. A creación dunha planta require seleccionar unha especie do catálogo local, asignar un nome personalizado e indicar a data de plantación. O catálogo de especies mantense separado das plantas concretas, evitando repetir a información común e permitindo unha relación relacional coherente.

10.5. Seguimento da evolución

O módulo de evolución permite engadir rexistros cronolóxicos a unha planta e consultar o seu historial. Cada rexistro pode incluír data, altura e notas. A persistencia depende de PlantEvolutionRecordRepository e os datos elimínanse en cascada se desaparece a planta á que pertencen.

10.6. Layout Designer

O Layout Designer representa visualmente as plantas dunha horta nun taboleiro. LayoutBuilder proporciona as dimensións dispoñibles e Stack/Positioned permiten situar cada elemento. GestureDetector xestiona o arrastre: durante onPanUpdate a posición modifícase localmente para manter unha interacción fluída e en onPanEnd persístese a posición final.

As coordenadas son normalizadas e limítanse mediante clamp para manter os elementos dentro do taboleiro. Tamén se comproba o solapamento antes de aceptar unha nova posición. Durante as probas detectouse que unha planta retirada e engadida de novo podía recibir unha posición inicial ocupada; a inserción modificouse para buscar previamente unha posición dispoñible. A persistencia foi comprobada tras navegar e reiniciar a aplicación.

10.7. Xestión de tarefas

O módulo de tarefas foi incorporado como unha ampliación pequena ao final do MVP, reutilizando a arquitectura existente. Inclúe o modelo Task, o contrato TaskRepository, unha implementación SQLite, TasksViewModel, TasksScreen e CreateTaskScreen. Permite crear, consultar, completar e eliminar tarefas persistentes, mantendo o seu estado básico, sen introducir nesta fase relacións adicionais con hortas ou plantas.

10.8. Persistencia e composición da aplicación

main.dart actúa como punto de composición. Nel créanse DatabaseService, os servizos HTTP, as implementacións concretas dos repositorios e os ViewModels proporcionados mediante MultiProvider. Todos os repositorios SQLite comparten a mesma instancia de DatabaseService, garantindo unha única configuración coherente da base de datos.

10.9. Evidencias de implementación

As Figuras 3 a 12 documentan os módulos implementados sen duplicar capturas neste capítulo. O diagrama E/R da Figura 2 e o esquema arquitectónico da Figura 1 complementan estas evidencias desde o punto de vista técnico.

11. Probas e validación

11.1. Estratexia de validación

A validación realizada durante o desenvolvemento foi principalmente manual e incremental. Cada módulo foi probado ao incorporarse e volveuse comprobar despois de cambios que podían afectar á navegación, ao estado compartido ou á persistencia. Non se presenta unha batería de tests automatizados que non forme parte do traballo realmente realizado; a memoria documenta as comprobacións efectivamente executadas.

11.2. Comprobacións funcionais

As comprobacións funcionais abrangueron os fluxos principais de hortas, plantas, evolución, meteoroloxía, deseño e tarefas. As operacións foron repetidas durante o desenvolvemento e novamente revisadas tras os cambios finais na persistencia e na configuración da aplicación. A táboa de resultados recolle os casos representativos e o seu estado final.

Área	Comprobación	Resultado
Hortas	Creación, consulta, edición, eliminación e persistencia	Correcto
Plantas	CRUD, asociación con especie e persistencia	Correcto
Evolución	Alta e consulta de rexistros; campos opcionais	Correcto
Meteoroloxía	Consulta con coordenadas persistidas e xestión básica de erros	Correcto
Layout Designer	Engadir, mover, retirar, evitar solapamentos e conservar posición	Correcto
Tarefas	Creación, consulta, completado, eliminación e persistencia	Correcto
Navegación	Percorrido entre pantallas principais e retorno	Correcto
Responsive	Revisión manual en tamaños de móbil, tablet e escritorio	Correcto no rango obxectivo
Análise estática	Execución final de flutter analyze	Sen incidencias

11.3. Persistencia e integridade

A persistencia foi comprobada pechando e volvendo abrir a aplicación e verificando que hortas, plantas, rexistros, disposición e tarefas conservaban os seus datos. Durante a evolución do esquema tamén se verificou a conservación de información ao aplicar migracións. As relacións CASCADE e RESTRICT foron utilizadas para manter a coherencia entre entidades dependentes.

11.4. Validación da interface

As probas de tamaño incluíron diferentes anchos de ventá e formatos de dispositivo. O comportamento foi considerado adecuado para os tamaños obxectivo. O caso de overflow observado por baixo de aproximadamente 314 píxeles lóxicos queda documentado como limitación coñecida e non como erro bloqueante da versión final.

11.5. Análise estática e compilación

Na revisión final executouse flutter analyze sobre o proxecto e o resultado foi “No issues found”. Previamente retirouse o test de exemplo xerado automaticamente por Flutter, xa incompatible co construtor actual de MartolaApp e sen valor como proba do comportamento implementado. Tamén se verificaron compilacións Release para Windows x64 e Android empregando a configuración externa da API mediante `--dart-define-from-file=config/secrets.json`. A build de Windows xerou MARTOLA.exe e a build Android xerou app-release.apk (54,6 MB). Un intento anterior da compilación Windows fallara porque o executable estaba aberto; ao pechar a aplicación, a compilación completouse correctamente.

A compilación Linux non foi verificada, xa que a build de escritorio para Linux debe realizarse nun ambiente Linux. Por rigor, esta plataforma mantense como obxectivo compatible previsto, pero non se presenta como compilación comprobada nesta entrega.

11.6. Incidencias relevantes e resolución

Durante o peche do proxecto apareceron varias incidencias útiles para validar o proceso. O test de exemplo de Flutter quedara desactualizado respecto das dependencias obrigatorias de MartolaApp e provocaba erros en flutter analyze; ao non formar parte da estratexia real de probas, eliminouse e a análise quedou limpa.

A primeira build Release de Windows fallou porque MARTOLA.exe permanecía aberto e Windows bloqueaba os ficheiros que CMake debía instalar. Tras pechar a aplicación, a compilación completouse correctamente.

Tamén se detectou a duplicación dos datos iniciais de plant_species. A causa estaba na inicialización concorrente da base de datos durante o arranque. A solución consistiu en compartir o proceso de apertura para evitar dúas inicializacións simultáneas e manter o seeding condicionado a unha táboa baleira.

Finalmente, a base de datos foi trasladada desde o directorio de documentos do usuario ao directorio de soporte da propia aplicación. Esta decisión separa os datos internos dos documentos persoais e resulta máis apropiada para unha aplicación de escritorio.

11.7. Resultado final e distribución

Tras as correccións finais, a versión 1.0.0 superou a análise estática de Flutter sen incidencias. A compilación Release de Windows x64 xerou correctamente MARTOLA.exe e a compilación Release de Android xerou a APK correspondente. Ambas builds recibiron a configuración de OpenWeather mediante `--dart-define-from-file=config/secrets.json`.

Os artefactos distribuíbles publicáronse nunha Release v1.0.0 de GitHub: un paquete ZIP para Windows x64 que conserva o executable, as DLL e o directorio de datos necesarios; unha APK Android; e un vídeo de demostración do funcionamento. A compilación Linux queda identificada

como compatible prevista, pero non verificada nesta entrega ao requirir un ambiente Linux para realizar a build.

12. Orzamento e viabilidade

A elaboración dun orzamento permite estimar o valor económico que tería o desenvolvemento de MARTOLA fóra do contexto académico. Esta estimación non representa o gasto real efectuado durante o TFC, xa que se utilizaron ferramentas gratuítas ou xa dispoñibles e o traballo de desenvolvemento foi realizado polo propio alumno.

Para obter unha referencia cuantitativa utilízase unha estimación conservadora de 120 horas de traballo e unha tarifa orientativa de 25 € por hora. O número de horas agrupa análise, deseño, aprendizaxe específica, implementación, probas, corrección de erros, documentación e preparación da versión entregable. A tarifa utilízase unicamente como valor de referencia para aproximar o custo dun desenvolvemento equivalente realizado profesionalmente.

12.1. Estimación de esforzo

Área de traballo	Horas estimadas
Análise, requisitos e planificación	12 h
Deseño de datos, arquitectura e interface	16 h
Aprendizaxe e preparación de Flutter/Dart	15 h
Implementación funcional	52 h
Integración de SQLite e APIs externas	10 h
Probas, correccións e adaptación responsive	8 h
Documentación e preparación da entrega	7 h
Total	120 h

12.2. Estimación económica

Concepto	Estimación
Desenvolvemento (120 h × 25 €/h)	3.000 €
Software de desenvolvemento	0 €
Control de versións e repositorio	0 €
OpenWeather no alcance utilizado	0 €
Amortización orientativa de equipamento	100 €
Consumos e conectividade asociados	40 €
Total estimado	3.140 €

As ferramentas fundamentais empregadas no proxecto —Flutter, Dart, VS Code, Git, GitHub e as librarías utilizadas— non supuxeron un custo directo para o desenvolvemento. O uso de OpenWeather realizouse dentro das condicións dispoñibles para o alcance do proxecto. Por este motivo, o principal custo económico dunha aplicación deste tipo está asociado ao tempo de análise e desenvolvemento.

12.3. Viabilidade

Desde o punto de vista técnico, a viabilidade de MARTOLA queda demostrada pola existencia dun MVP funcional cunha arquitectura reutilizable en Android e escritorio. O enfoque Local First reduce a necesidade de infraestrutura de servidor e permite que a maior parte das funcionalidades continúen dispoñibles sen depender dunha conexión permanente.

Desde o punto de vista económico, a utilización de tecnoloxías de código aberto ou sen custo de licenza mantén baixos os gastos de infraestrutura durante as primeiras fases. Unha eventual evolución cara a sincronización cloud, contas de usuario, notificacións ou servizos meteorolóxicos de maior capacidade introduciría custos adicionais de backend, almacenamento, mantemento e posibles plans de API.

En consecuencia, MARTOLA resulta viable como proxecto académico e como prototipo funcional. A súa conversión nun produto comercial requiriría un estudo adicional de usuarios potenciais, modelo de negocio, soporte, privacidade, seguridade e custos recorrentes.

13. Aspectos innovadores e valor achegado

O valor principal de MARTOLA non reside nunha funcionalidade illada completamente inédita, senón na integración coherente de diferentes necesidades relacionadas coa xestión dunha horta dentro dunha mesma aplicación multiplataforma.

A aplicación combina xestión de hortas e plantas, seguimento histórico da evolución, información meteorolóxica contextualizada, xeocodificación, tarefas e representación espacial das plantas. Esta combinación permite que información que habitualmente podería estar distribuída entre anotacións, aplicacións meteorolóxicas ou ferramentas de planificación se xestione desde un único fluxo.

13.1. Enfoque Local First

A persistencia principal realízase localmente mediante SQLite. Isto simplifica o funcionamento, reduce dependencias externas e permite manter operativas as funcións principais mesmo cando non existe conectividade. As APIs externas utilízanse só para os datos que realmente requiren unha fonte remota, como a meteoroloxía e a xeocodificación.

13.2. Deseñador visual persistente

Un dos elementos máis diferenciais do MVP é o Layout Designer. As plantas dunha horta poden incorporarse a un taboleiro, desprazarse mediante interacción directa e conservar a súa posición. As coordenadas almacénanse normalizadas, polo que a disposición pode reconstruírse en superficies de diferente tamaño. Ademais, implementáronse límites, prevención de solapamentos e busca dunha posición inicial libre.

13.3. Arquitectura preparada para evolución

A separación mediante MVVM simplificado, Repository Pattern e Services facilita que determinadas decisións de infraestrutura poidan evolucionar sen obrigir a redeseñar toda a interface. Por exemplo, a capa de presentación non depende directamente de OpenWeather nin de SQLite, senón dos contratos e ViewModels correspondentes.

13.4. Desenvolvemento multiplataforma

Flutter permite manter unha base de código compartida para dispositivos móbiles e escritorio. Este enfoque é especialmente relevante nun proxecto deste tipo, no que unha horta pode consultarse desde un teléfono durante o traballo no exterior e desde un equipo de escritorio para tarefas de organización ou revisión. A revisión final confirmou este enfoque mediante builds Release correctas para Windows x64 e Android a partir da mesma base de código.

En conxunto, o valor achegado de MARTOLA está na combinación de funcionalidades, na organización técnica do proxecto e na capacidade de servir como base extensible para futuras funcionalidades máis avanzadas.

14. Conclusións

O desenvolvemento de MARTOLA permitiu completar unha aplicación multiplataforma funcional partindo dunha idea inicial ampla e transformándoa progresivamente nun alcance realizable. A definición dun MVP foi fundamental para priorizar as funcionalidades e evitar que ampliacións secundarias impedisen pechar os módulos principais.

Os obxectivos fundamentais considéranse alcanzados: a aplicación permite xestionar hortas, plantas e rexistros de evolución; dispón de persistencia SQLite; integra xeocodificación e información meteorolóxica; incorpora un deseñador visual persistente e un módulo básico de tarefas; e mantén unha arquitectura separada por responsabilidades.

Un dos aspectos máis relevantes do proxecto foi a evolución técnica. O desenvolvemento non se limitou á creación das pantallas, senón que introduciu gradualmente conceptos como xestión de estado con Provider, arquitectura MVVM simplificada, Repository Pattern, inxección de dependencias, programación asíncrona, consumo de APIs REST, migracións SQLite, integridade referencial e interfaces adaptables.

O proceso incremental tamén permitiu detectar e resolver problemas reais durante o desenvolvemento. Entre eles destacan a conservación dos datos durante as migracións, a actualización correcta do estado da interface, a asociación da meteoroloxía ás coordenadas dunha horta, a persistencia das posicións no Layout Designer, a prevención de solapamentos e os problemas de overflow en determinados tamaños de pantalla.

As probas realizadas e o resultado de flutter analyze sen incidencias proporcionan unha base razoable para considerar a versión actual estable dentro do alcance definido para o MVP. As builds Release verificadas para Windows x64 e Android confirman ademais que a aplicación pode executarse fóra do contorno habitual de desenvolvemento e reforzan o carácter multiplataforma do proxecto.

Ao mesmo tempo, o proxecto deixa claras as súas limitacións. A versión actual non dispón de autenticación, sincronización entre dispositivos nin histórico meteorolóxico persistente, e determinados aspectos visuais poderían recibir un maior nivel de refinamento. Estas limitacións responden principalmente á priorización do alcance e non impiden cumprir os obxectivos principais do MVP.

En conclusión, MARTOLA constitúe unha base funcional e extensible sobre a que poder continuar traballando. O proxecto permitiu integrar nun caso práctico diferentes áreas propias do desenvolvemento de aplicacións multiplataforma e ofrece un resultado demostrable tanto desde o punto de vista funcional como arquitectónico.

O desenvolvemento de MARTOLA supuxo un proceso de aprendizaxe importante, especialmente por tratarse da primeira aplicación completa desenvolvida polo autor. Unha das principais dificultades iniciais foi a adaptación a Flutter e, en particular, ao seu modelo de construción da interface baseado en widgets, un concepto diferente aos empregados anteriormente. A isto sumouse a necesidade de comprender como se comunican as distintas capas da aplicación e como Provider permite xestionar e propagar o estado entre elas.

Un dos aspectos máis satisfactorios do proxecto foi a construción da arquitectura da aplicación. A separación entre vistas, ViewModels, repositorios, base de datos e servizos externos permitiu comprender de maneira práctica como organizar unha aplicación máis alá de exercicios ou proxectos de menor tamaño. Ata este proxecto non se realizara de forma completa unha estrutura deste tipo, polo que a súa implementación constitúe unha das aprendizaxes máis relevantes.

O proxecto tamén permitiu adquirir experiencia na organización e documentación continuada dun desenvolvemento, na integración de APIs externas e na evolución dunha base de datos mediante diferentes versións e migracións. A linguaxe Dart non supuxo unha dificultade especialmente elevada debido á súa similitude con outras linguaxes estudadas previamente; pola contra, o paradigma de Flutter baseado en widgets e a organización da aplicación arredor deles representaron unha das principais novidades.

O resultado actual non se considera un produto pechado. Entre as ampliacións que se consideran máis interesantes están a integración con MeteoSIX para proporcionar unha previsión meteorolóxica máis completa, a incorporación dun histórico meteorolóxico e a mellora da interface, especialmente no deseñador de hortas, buscando unha identidade visual máis persoal. Tamén se contempla incorporar autenticación de usuarios, aínda que se considera unha ampliación menos prioritaria que as anteriores.

A valoración final do proxecto é positiva. Tendo en conta o punto de partida, conseguir unha aplicación completamente funcional, aínda que deliberadamente sinxela, representa un resultado satisfactorio. MARTOLA é a primeira aplicación completa desenvolvida polo autor e, máis que considerarse un produto rematado, constitúe unha base sobre a que continuar traballando. O coñecemento adquirido durante o seu desenvolvemento permite afrontar as futuras ampliacións cunha comprensión moito maior da estrutura e do ciclo de vida dun proxecto de software.

15. Liñas futuras

A arquitectura e o modelo de datos de MARTOLA permiten formular diferentes ampliacións que quedaron fóra do alcance do MVP. Estas propostas non se consideran requisitos pendentes da versión actual, senón posibles direccións de evolución do proxecto.

15.1. Autenticación e xestión de usuarios

A incorporación dun sistema de autenticación permitiría diferenciar contas e asociar cada horta ao seu propietario. Esta funcionalidade sería un paso previo á sincronización entre dispositivos e ao uso compartido de información.

15.2. Sincronización cloud

Un backend remoto permitiría sincronizar os datos locais, realizar copias de seguridade e acceder á mesma información desde varios dispositivos. O enfoque Local First actual podería conservarse, engadindo sincronización sobre a persistencia local.

15.3. Histórico meteorolóxico

O modelo obxectivo xa contempla a posibilidade de almacenar rexistros meteorolóxicos. Unha ampliación futura podería gardar observacións ou predicións periódicas para analizar a evolución das condicións dunha horta e relacionalas co crecemento das plantas. Tamén se contempla estudar a incorporación de MeteoSIX como fonte meteorolóxica adicional ou alternativa, especialmente para información de ámbito galego.

15.4. Catálogo vexetal ampliado

O catálogo local actual é deliberadamente reducido. Podería ampliarse con información de rega, exposición solar, tipo de solo, rangos de temperatura, épocas de plantación e outros datos agronómicos, ben mediante datos propios ou unha fonte externa.

15.5. Fotografías e evolución avanzada

Os rexistros de evolución poderían incorporar fotografías, estados de crecemento, incidencias, floración ou frutificación. Isto permitiría construír un histórico visual máis completo de cada planta.

15.6. Tarefas, recordatorios e notificacións

O módulo básico de tarefas podería evolucionar cara a datas límite, prioridades, recorrencias, asociación con hortas ou plantas concretas e notificacións locais.

15.7. Melloras do Layout Designer

Entre as posibles melloras atópanse o encaixe nunha cuadrícula, representación específica segundo a especie, elementos de diferente tamaño, zonas de cultivo, medidas reais ou edición máis avanzada da superficie.

15.8. Estatísticas e recomendacións

A acumulación de datos permitiría introducir gráficos de evolución, resumos por horta, estatísticas de tarefas ou recomendacións baseadas na especie, época do ano e condicións meteorolóxicas.

15.9. Versión web e distribución

Flutter permite valorar no futuro unha versión web. A versión 1.0.0 xa dispón de artefactos distribuíbles básicos para Windows x64 e Android mediante GitHub Releases; como evolución poderían prepararse instaladores específicos para escritorio, publicación en tendas e mecanismos máis robustos de configuración e protección das credenciais de servizos externos.

16. Bibliografía e recursos

As seguintes referencias corresponden á documentación oficial e ás fontes empregadas para contextualizar as tecnoloxías e o estudo de mercado. Data de consulta: 26 de agosto de 2026.

16.1. Estudo de mercado

[1] Planter. “Planter | Garden Planner” e documentación “Getting Started”. <https://planter.garden/> ; <https://planter.garden/docs/getting-started/>

[2] From Seed to Spoon. “The Seed to Spoon Garden Planner App” e “App Highlights”. <https://www.seedtospoon.net/> ; <https://www.seedtospoon.net/app/>

[3] GrowVeg. “Garden Planner User Guide”. <https://www.growveg.com/downloads/gardenplanneruserguidev3.pdf>

[4] Gardenize. “Gardening journal and planner for desktop, mobile and tablet”. <https://gardenize.com/best-digital-garden-journal-app/>

16.2. Tecnoloxías principais

[5] Flutter. “Supported deployment platforms”. <https://docs.flutter.dev/reference/supported-platforms>

[6] Flutter. “Build for and integrate with multiple platforms”. <https://docs.flutter.dev/platform-integration>

[7] Flutter. “Flutter architectural overview”. <https://docs.flutter.dev/resources/architectural-overview>

[8] Provider. Paquete oficial publicado en pub.dev. <https://pub.dev/packages/provider>

[9] sqflite. Plugin SQLite para Flutter publicado en pub.dev. <https://pub.dev/packages/sqflite>

[10] sqflite_common_ffi. Implementación FFI de SQLite para Dart/Flutter. https://pub.dev/packages/sqflite_common_ffi

[11] path_provider. Plugin para obter localizacións habituais do sistema de ficheiros. https://pub.dev/packages/path_provider

[12] http. Biblioteca HTTP para Dart. <https://pub.dev/packages/http>

[13] OpenWeather. Documentación das APIs de Current Weather Data e Geocoding. <https://openweathermap.org/api> ; <https://openweathermap.org/api/geocoding-api>

[14] SQLite. Documentación oficial. <https://www.sqlite.org/docs.html>

[15] Git. Documentación oficial. <https://git-scm.com/doc>

[16] GitHub Docs. Documentación oficial de GitHub. <https://docs.github.com/>

17. Anexos

Os anexos complementarios e os materiais distribuíbles mantéñense no repositorio e na Release v1.0.0 do proxecto. A memoria integra directamente as capturas e diagramas máis relevantes para evitar duplicacións innecesarias.

17.1. Repositorio e documentación técnica

O repositorio contén o código fonte Flutter, a documentación técnica elaborada durante o desenvolvemento e o directorio de diagramas. Consérvase tanto o deseño inicial da base de datos como o modelo final v6, permitindo observar a evolución entre a planificación e a implementación.

17.2. Release v1.0.0

A Release v1.0.0 inclúe un paquete ZIP para Windows x64 coa distribución completa necesaria para executar MARTOLA, unha APK Android e un vídeo de demostración do funcionamento. Estes ficheiros non se incorporan ao historial normal de Git para manter o repositorio centrado no código fonte e na documentación.

17.3. Configuración e seguridade da API

A clave de OpenWeather non se almacena no repositorio. O proxecto inclúe un ficheiro de exemplo e a configuración real mantense en `config/secrets.json`, excluído mediante `.gitignore`. As execucións e builds que necesitan os servizos remotos reciben os valores mediante `--dart-define-from-file`.

17.4. Manual de usuario

A versión 1.0.0 foi deseñada cun fluxo de uso directo. Ao iniciar MARTOLA móstrase a pantalla de benvida; o botón Entrar conduce ao Dashboard, desde o que se accede ás hortas, ás tarefas e ás accións rápidas.

17.4.1. Xestión de hortas

Desde As miñas hortas pódese crear unha nova horta co botón de engadir. O formulario solicita nome, localización e superficie. A opción Buscar localización consulta o servizo de xeocodificación; ao seleccionar un resultado gárdanse tamén as coordenadas. Ao abrir unha horta existente móstranse os seus datos, a meteoroloxía actual cando hai coordenadas e as accións para editar, deseñar, consultar plantas ou eliminar.

17.4.2. Plantas e evolución

A Lista de plantas mostra os exemplares asociados á horta. O botón de engadir permite indicar un nome personalizado, seleccionar unha especie do catálogo e establecer a data de plantación. Desde o detalle dunha planta pódese acceder aos rexistros de evolución, nos que se gardan a data e, opcionalmente, a altura e notas. Os rexistros poden consultarse, editarse ou eliminarse.

17.4.3. Deseño da horta

A opción Deseñar horta abre o Layout Designer. As plantas aínda non colocadas poden seleccionarse no despregable e engadirse ao taboleiro. Cada elemento pode arrastrarse para modificar a súa posición. Unha pulsación longa permite retiralo do deseño sen eliminar a planta da horta. As posicións quedan persistidas e recupéranse ao volver abrir a pantalla.

17.4.4. Tareas

Desde o Dashboard pódese abrir o listado de tarefas ou crear unha mediante Accións rápidas. As tarefas poden incluír título, descrición e data límite. O usuario pode marcalas como completadas ou eliminalas; o Dashboard mostra o número de tarefas pendentes.

17.5. Manual técnico e de instalación

17.5.1. Requisitos para desenvolvemento

Para executar o proxecto desde o código fonte é necesario dispoñer dun SDK de Flutter compatible co proxecto, as ferramentas da plataforma de destino e as dependencias declaradas en pubspec.yaml. Desde o directorio app/ deben instalarse as dependencias mediante flutter pub get.

17.5.2. Configuración de OpenWeather

O repositorio non contén a API key real. Debe crearse localmente config/secrets.json tomando como referencia config/secrets.example.json. A aplicación pode executarse con flutter run --dart-define-from-file=config/secrets.json. O ficheiro real permanece excluído do control de versións mediante .gitignore.

17.5.3. Instalación en Windows

A Release inclúe un ZIP para Windows x64. Debe descomprimirse completo, mantendo MARTOLA.exe, flutter_windows.dll, sqlite3.dll e o directorio data no mesmo conxunto. A aplicación execútase abrindo MARTOLA.exe; non debe copiarse unicamente o executable porque depende dos demais ficheiros da distribución.

17.5.4. Instalación en Android

A Release inclúe a APK da versión 1.0.0. Pode transferirse a un dispositivo Android e instalarse como aplicación externa. Segundo a configuración do sistema, Android pode solicitar permiso para instalar aplicacións procedentes desa fonte. A APK entregada corresponde a unha build Release.

17.5.5. Comandos de validación e build

A análise estática final realizouse con flutter analyze. As builds verificadas utilizáronse cos comandos flutter build windows --release --dart-define-from-file=config/secrets.json e flutter build apk --release --dart-define-from-file=config/secrets.json. A compilación para Linux non se verificou por requirir un ambiente Linux.